lista1, sprawozdanie

Dawid Marszałek

October 2026

1 Wprowadzenie

Celem niniejszego laboratorium jest analiza liczby operacji dominujacych wykonywanych przez wybrane algorytmy sortowania w zależności od rozmiaru danych wejściowych N. Badanie pomoże wyłonić najlepszy (czyli z najmniejsza ilościa przypisań i porównań) algorytm do sortowania liczb w tablicach.

W ramach badań przeanalizowano i porównano 5 algorytmów:

- **Insertion Sort** – klasyczne sortowanie przez wstawianie,
- **Pairwise Insertion Sort** – zoptymalizowany wariant sortowania przez wstawianie elementów parami,
- **Bubble Sort** – sortowanie babelkowe z wykrzykiem braku zamian,
- **Merge Sort (2-way)** – klasyczne sortowanie przez scalanie (dwie podtablice),
- **Merge Sort (3-way)** – zoptymalizowane sortowanie przez scalanie (trzy-podtablice).

Pomiary zostały przeprowadzone na z góry ustalonych, nieposortowanych ciagach liczbowych o rozmiarach $N \in \{10, 15, 20\}$.

2 Dane Testowe i Wykresy Pomiarów

Do przeprowadzenia badań użyto trzech stałych, nieposortowanych wektorów testowych o rozmiarach $N \in \{10, 15, 20\}$:

- **Wektor** $N = 10$**:** {`42, 15, 88, 3, 27, 91, 14, 63, 50, 5`}
- **Wektor** $N = 15$**:** {`71, 12, 45, 99, 3, 28, 64, 82, 19, 33, 50, 7, 91, 24, 60`}
- **Wektor** $N = 20$**:** {`84, 19, 62, 3, 91, 47, 15, 73, 28, 55, 9, 36, 68, 94, 12, 41, 80, 23, 51, 30`}

Na podstawie uzyskanych wyników wygenerowano wykresy porównawcze zaprezentowane na rysunkach 1 oraz 2.

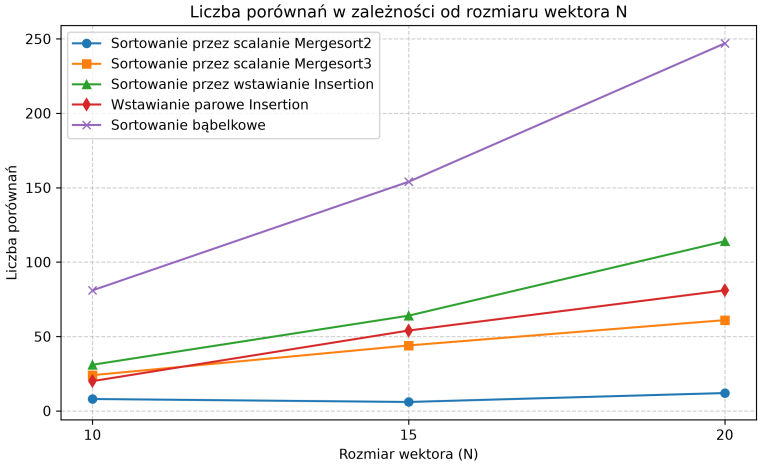


Figure 1: Liczba porównań w zależności od rozmiaru wektora N.

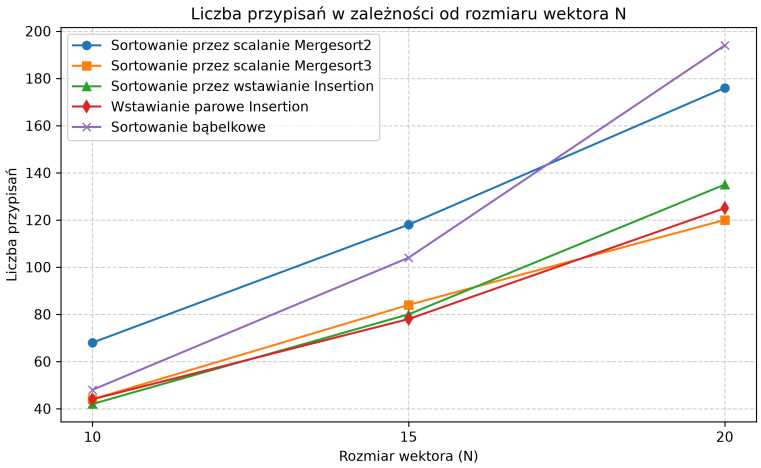


Figure 2: Liczba przypisań w zależności od rozmiaru wektora N.

3 Merge Sort

Merge Sort jest sortowaniem przez scalanie o złożoności $O(N \log N)$. Jak widać na powyższych wykresach, Merge Sort (2-way) uzyskał najniższa liczbe porównań. Wynika to bezpośrednio z faktu, że porównanie wartości elementów z wektora odbywa sie tylko w jednym miejscu petli scalajacej:

```
// Porownanie zachodzi wylacznie wtedy, gdy oba indeksy sa
    prawidlowe
if (i < length1 && j < length2) {
    porownanie++;
}

if ((i < length1) && ((j >= length2) || (vL[i] <= vR[j]))) {
    v[k] = vL[i];
    przypisanie++;
    i++;
} else {
    v[k] = vR[j];
    przypisanie++;
    j++;
}
```

Z drugiej strony można zauważyć, że liczba przypisań jest najwyższa dla wektorów 10,15-elementowych, widocznie odbiegajac od innych sortowań z wyłaczeniem sortowania babelkowego. Tak prezenetuja sie wyniki dla Merge Sort (2-way):

Rozmiar wektora (N)	**Porównania**	**Przypisania**
10	8	68
15	6	118
20	12	176

Na podstawie uzyskanych wyników można wywnioskować, że wraz ze wzrostem rozmiaru wektora o 5 elementów liczba przypisań wzrasta o około 55 operacji, podczas gdy liczba porównań pozostaje na bardzo niskim poziomie.

Proces sortowania algorytmem **Merge Sort (2-way)** przebiega w dwóch głównych etapach:

1. **Dzielenie:** Tablica wejściowa jest rekurencyjnie dzielona na dwie równe cześci (lub różniace sie rozmiarem o jeden element w przypadku nieparzystej długości). Proces podziału powtarza sie do momentu uzyskania podtablic jednoelementowych.

2. **Scalanie:** Dedykowana funkcja pomocnicza scala ze soba wcześniej posortowane podtablice, porównujac ich kolejne elementy. Działanie rozpoczyna sie od tablic jednoelementowych (które z definicji sa posortowane) i trwa rekurencyjnie do momentu odtworzenia pełnej, n-elementowej tablicy wyjściowej.

3.1 Złożoność obliczeniowa

Złożoność czasowa algorytmu **Merge Sort** wynosi $\mathcal{O}(N \log N)$ we wszystkich trzech przypadkach: pesymistycznym, średnim oraz optymistycznym. Złożoność pamieciowa wynosi natomiast $\mathcal{O}(N)$.

Co z tego wynika w praktyce?

- **Niezależność od układu danych:** Czas wykonania algorytmu oraz liczba operacji nie zależa od tego, czy wejściowa tablica jest już cześciowo posortowana, czy zupełnie losowa. Struktura podziałów i scalania zawsze pozostaje taka sama.
- **Wysoka skalowalność dla dużych N:** Wzrost funkcji $N \log N$ jest znacznie wolniejszy niż funkcji kwadratowej N^2. O ile dla małych wektorów ($N \in \{10, 15, 20\}$) różnice w czasie i liczbie operacji sa nieznaczne, o tyle dla dużych zbiorów danych Merge Sort deklasuje algorytmy kwadratowe (np. Bubble Sort).
- **Narzut pamieciowy:** Gwarancja złożoności $\mathcal{O}(N \log N)$ wymaga utworzenia dodatkowych podtablic pomocniczych w etapie scalania. Koszt ten uwidacznia sie w uzyskanych wynikach pomiarów w postaci wyższej liczby przypisań pamieci w porównaniu do algorytmów działajacych w miejscu (in-place).

4 Merge Sort (3-way)

Algorytm **Merge Sort (3-way)** jest bezpośrednia modyfikacja wersji dwudrożnej. Główna różnica polega na tym, że na każdym etapie rekurencji wektor wejściowy jest dzielony nie na dwie, lecz na trzy w miare równe podtablice (`vL`, `vM`, `vR`). W konsekwencji etap scalania musi połaczyć ze soba trzy posortowane ciagi w jeden.

4.1 Analiza operacji porównań

Zwiekszenie liczby podtablic do trzech wywołuje wzrost liczby porównań w etapie scalania. Aby wyłonić najmniejszy element spośród trzech kandydatów, algorytm musi w najgorszym przypadku wykonać dwa porównania na każda pozycje wynikowa.

Poniższy fragment kodu przedstawia logike petli scalajacej z inkrementacja liczników operacji dominujacych:

```
int i = 0, j = 0, m = 0;
for (int k = left; k <= right; k++) {
    // Sprawdzamy, ktora z trzech podtablic ma najmniejszy
       element
    if ((i < length1) && (j >= length2 || (++porownanie &&
       vL[i] <= vM[j])) && (m >= length3 || (++porownanie &&
        vL[i] <= vR[m]))) {
        v[k] = vL[i];
        i++;
        przypisanie++;
    } else if ((j < length2) && (m >= length3 || (++
       porownanie && vM[j] <= vR[m]))) {
        v[k] = vM[j];
        j++;
        przypisanie++;
    } else {
        v[k] = vR[m];
        m++;
        przypisanie++;
    }
}
```

Jest to niezwykle ciekawy fragment kodu, ponieważ pokazuje, w jaki sposób złożona logika spójników logicznych oraz operatorów skróconego wartościowania (`&&`, `||`) w warunkach `if` bezpośrednio wymusza wieksza liczbe porównań przy wyłanianiu najmniejszego elementu z trzech podtablic jednocześnie.

4.2 Bilans: Wieksza liczba porównań vs Mniejsza liczba przypisań

Mimo że liczba porównań w wersji 3-way wyraźnie wzrasta (np. dla $N = 20$: 61 porównań w 3-way wobec 12 w 2-way), algorytm ten uzyskuje wyraźnie mniejsza

liczbe przypisań (dla $N = 20$: 120 przypisań w 3-way wobec 176 w 2-way).

Wynika to z faktu, że podział na 3 cześci zmniejsza wysokość drzewa rekurencji z $\log_2 N$ do $\log_3 N$. Płatsze drzewo rekurencji oznacza mniej poziomów scalania, a co za tym idzie — rzadsze przepisywanie elementów pomiedzy wektorem głównym a tablicami pomocniczymi.

4.3 Wyniki pomiarów

Pomiary przeprowadzone dla algorytmu **Merge Sort (3-way)** dały nastepujace rezultaty:

- $N = 10$**:** 24 porównania, 44 przypisania
- $N = 15$**:** 44 porównania, 84 przypisania
- $N = 20$**:** 61 porównań, 120 przypisań

4.4 Bilans: Wieksza liczba porównań vs Mniejsza liczba przypisań

Mimo że liczba porównań w wersji 3-drożnej wyraźnie wzrasta (np. dla $N = 20$: 61 porównań w 3-way wobec 12 w 2-way), algorytm ten uzyskuje **wyraźnie mniejsza liczbe przypisań** (dla $N = 20$: 120 przypisań w 3-way wobec 176 w 2-way).

4.5 Złożoność obliczeniowa i porównanie z wersji 2-way

- **Złożoność czasowa:** Wynosi $\mathcal{O}(N \log_3 N)$, co asymptotycznie sprowadza sie do tej samej klasy złożoności $\mathcal{O}(N \log N)$ co w Merge Sort 2-way.
- **Złożoność pamieciowa:** Pozostaje na poziomie $\mathcal{O}(N)$, ponieważ suma rozmiarów trzech podtablic pomocniczych na danym poziomie rekurencji odpowiada rozmiarowi scalanego fragmentu.

Podsumowanie różnic wzgledem Merge Sort (2-way):

1. **Głebia rekurencji:** Drzewo wywołań jest niższe ($\log_3 N < \log_2 N$), co zmniejsza narzut na wywołania funkcji oraz liczbe operacji zapisu/kopiowania do pamieci.
2. **Złożoność warunków w petli:** Scalanie wymaga bardziej złożonej logiki branchingowej (`if-else`), co zwieksza stała kosztem liczby porównań, ale oszczedza ruch w pamieci RAM.

5 Insertion Sort

Algorytm **Insertion Sort** (sortowanie przez wstawianie) polega na stopniowym budowaniu posortowanego ciagu poprzez wstawianie kolejnych elementów na ich właściwe miejsca. Działanie algorytmu opiera sie na dwóch głównych etapach:

1. **Wyszukiwanie pozycji dla klucza:** Pobieramy kolejny element z nieposortowanej cześci tablicy (nazywany kluczem" – `key`) o indeksie i i porównujemy go z elementami po lewej stronie (zaczynajac od indeksu $i-1$), przesuwajac sie w lewo, dopóki nie znajdziemy elementu mniejszego badź równego.
2. **Przesuwanie elementów i wstawienie:** Wszystkie elementy wieksze od klucza sa przesuwane o jedna pozycje w prawo, tworzac wolne miejsce. Gdy petla sie zatrzyma, umieszczamy nasza wartość `key` na indeksie $+1$ wzgledem znalezionego mniejszego elementu.

5.1 Wyniki pomiarów

Pomiary przeprowadzone dla algorytmu Insertion Sort dały nastepujace rezultaty:

- $N = 10$**:** 31 porównań, 42 przypisania
- $N = 15$**:** 64 porównania, 80 przypisań
- $N = 20$**:** 114 porównań, 135 przypisań

5.2 Analiza i wnioski

Mimo że bezpośrednie porównanie wartości z kluczem wystepuje w kodzie tylko w jednym miejscu – w warunku petli `while`:

```
1 while (i >= 0 && (++porownanie && v[i] > key)) {
```

to łaczna liczba porównań (114 dla $N = 20$) jest znaczaco wyższa niż w przypadku obu wariantów Merge Sort oraz zoptymalizowanego Pairwise Insertion. Wynika to z faktu, że przy wstawianiu każdego nowego elementu musimy wielokrotnie sprawdzać kolejne pozycje wewnatrz już posortowanej podtablicy.

Liczba przypisań (135 dla $N = 20$) rośnie stosunkowo szybko ze wzgledu na konieczność przesuwania każdego wiekszego elementu o jedna pozycje w prawo. Algorytm wypada jednak lepiej od Bubble Sorta, ponieważ nie wykonuje trzykrotnych przypisań przy zamianach (`std::swap`), lecz przesuwa elementy pojedynczo.

5.3 Złożoność obliczeniowa

- **Złożoność czasowa:** $\mathcal{O}(N^2)$ w przypadku średnim i pesymistycznym oraz $\mathcal{O}(N)$ w przypadku optymistycznym (gdy tablica jest już posortowana).

- **Złożoność pamieciowa:** $\mathcal{O}(1)$ – algorytm działa całkowicie w miejscu, nie wymagajac dodatkowej pamieci RAM.

6 Pairwise Insertion Sort

Algorytm **Pairwise Insertion Sort** stanowi udoskonalona, zoptymalizowana wersje klasycznego sortowania przez wstawianie. Zamiast wstawiać elementy pojedynczo, algorytm przetwarza je parami, co przebiega w nastepujacych krokach:

1. Pobieramy dwa kolejne elementy z nieposortowanej cześci i porównujemy je ze soba, wyznaczajac element wiekszy oraz mniejszy.
2. Najpierw szukamy odpowiedniego miejsca w posortowanej cześci tablicy dla elementu **wiekszego**.
3. Miejsce dla elementu **mniejszego** nie jest szukane od samego końca posortowanej podtablicy, lecz rozpoczyna sie od indeksu wyznaczonego przed chwila dla elementu wiekszego.

Warto zwrócić uwage na parzystość rozmiaru tablicy. Przy parzystej liczbie elementów po przetworzeniu wszystkich par na samym końcu pozostaje jeden nieposortowany element. Należy go wówczas wstawić do tablicy klasycznym, pojedynczym ruchem, co przedstawia poniższa końcówka kodu:

```
// Sprawdzamy parzystosc rozmiaru tablicy - obsluga
    pozostalego elementu
if (v.size() % 2 == 0) {
    int key = v.back();
    przypisanie++;
    int i = v.size() - 2;
    while (i >= 0 && v[i] > key) {
        v[i + 1] = v[i];
        przypisanie++;
        i--;
    }
    v[i + 1] = key;
    przypisanie++;
}
```

6.1 Wyniki pomiarów

Pomiary przeprowadzone dla Pairwise Insertion Sort dały nastepujace rezultaty:

- $N = 10$**:** 20 porównań, 44 przypisania
- $N = 15$**:** 54 porównania, 78 przypisań
- $N = 20$**:** 81 porównań, 125 przypisań

6.2 Analiza i wnioski

Wariant parowy utrzymuje zbliżona liczbe przypisań w stosunku do klasycznego Insertion Sorta (dla $N = 20$: 125 vs 135), uzyskujac przy tym znaczaca redukcje liczby porównań (81 vs 114). Przewaga ta jest szczególnie widoczna w pesymistycznych układach danych, ponieważ rozpoczynanie poszukiwań dla mniejszego elementu od pozycji wiekszego eliminuje zbedne, powtarzajace sie porównania w posortowanej cześci wektora.

6.3 Złożoność obliczeniowa

- **Złożoność czasowa:** $\mathcal{O}(N^2)$ – ze stała w liczbie porównań niższa niż w klasycznym Insertion Sorcie.
- **Złożoność pamieciowa:** $\mathcal{O}(1)$ – algorytm działa całkowicie w miejscu.

7 Bubble Sort

Algorytm **Bubble Sort** (sortowanie babelkowe) charakteryzuje sie najprostsza struktura i najbardziej przejrzystym kodem spośród wszystkich badanych metod.

7.1 Zasada działania

Proces sortowania przebiega według nastepujacych kroków:

1. Porównujemy element o indeksie i z elementem nastepnym o indeksie $i+1$.
2. Jeśli element o indeksie i jest wiekszy od elementu $i + 1$, dokonujemy ich zamiany miejscami.
3. Zwiekszamy indeks i o jeden i powtarzamy porównanie dla kolejnej pary.
4. Całe przejście przez tablice powtarzamy cyklicznie do momentu, aż cały ciag bedzie posortowany.

7.2 Wyniki pomiarów

Pomiary przeprowadzone dla algorytmu Bubble Sort dały nastepujace rezultaty:

- $N = 10$**:** 81 porównań, 48 przypisań
- $N = 15$**:** 154 porównania, 104 przypisania
- $N = 20$**:** 247 porównań, 194 przypisania

7.3 Analiza i wnioski

Prostota oraz wysoka powtarzalność schematu działania skutkuja najwyższa liczba porównań i przypisań spośród wszystkich analizowanych algorytmów. Liczby te gwałtownie wzrastaja wraz ze wzrostem rozmiaru tablicy N, co wynika bezpośrednio z kwadratowej charakterystyki wzrostu. Na wysoki wynik przypisań wpływa dodatkowo fakt, że każda zamiana elementów za pomoca funkcji `std::swap` wymaga 3 operacji zapisu w pamieci.

7.4 Złożoność obliczeniowa

- **Złożoność czasowa:** $\mathcal{O}(N^2)$ w przypadku średnim i pesymistycznym (oraz $\mathcal{O}(N)$ w przypadku optymistycznym przy użyciu flagi braku zamian).
- **Złożoność pamieciowa:** $\mathcal{O}(1)$ – algorytm działa całkowicie w miejscu.

8 Wnioski Końcowe i Podsumowanie

Przeprowadzone badania pozwoliły na wyciagniecie nastepujacych wniosków dotyczacych efektywności oraz obszarów zastosowań analizowanych algorytmów:

1. **Kompromis czasowo-pamieciowy (Time-Space Trade-off):**
   - **Bubble Sort** oraz **Insertion Sort** działaja w miejscu (*in-place*), osiagajac optymalna złożoność pamieciowa $\mathcal{O}(1)$. Mimo że Bubble Sort wykonuje drastycznie wiecej porównań i przypisań, nie wymaga żadnej dodatkowej pamieci RAM.
   - **Merge Sort** gwarantuje wysoka wydajność czasowa $\mathcal{O}(N \log N)$, lecz odbywa sie to kosztem złożoności pamieciowej $\mathcal{O}(N)$ ze wzgledu na konieczność alokacji podtablic pomocniczych.
2. **Optymalizacja Insertion Sorta (Wariant Parowy):** Pobieranie i wstawianie elementów parami (*Pairwise Insertion Sort*) okazało sie skuteczna optymalizacja — zredukowało liczbe porównań o około 30% w stosunku do klasycznego Insertion Sorta (dla $N = 20$: 81 vs 114), zachowujac jednocześnie zalete działania w miejscu.
3. **Merge Sort 2-way vs 3-way:** Podział na 3 podtablice w Merge Sort 3-way pozwolił zmniejszyć głebokość drzewa rekurencji, co przełożyło sie na znaczacy spadek liczby przypisań pamieci (dla $N = 20$: 120 przypisań dla wersji 3-way wobec 176 dla 2-way).